

The workspace: both repos, their remotes, and the rules that span them

This file was `CLAUDE.md` at the folder root until 2026-09-06. It moved into the repo when Syncthing was retired, because a file outside a pushed repo now reaches exactly one machine. The folder root keeps a short pointer to it.

Personal project. Fork of YARG (Yet Another Rhythm Game), plus a new self-hosted song server. Everything here is LGPL-3.0-or-later, matching upstream.

Goal

Further develop YARG: graphics, controller compatibility (prioritising official Rock Band and Guitar Hero instruments), and major backend functionality. Contributions should stay upstreamable where possible.

#1 priority: a Docker-compatible song server for the YARG client, portable to Raspberry Pi, macOS and Windows. Once server/client works, additional features are modular and enabled from a config menu in the server app.

Repos and where they push

Three repos live under this folder. All are personal, so they follow the personal chain.

Folder	GitLab (origin, source of truth)	Public mirror
`yarg-song-server/`	`gitlab.badassium.com/fatalexception/yarg-song-server` (project 53)	`github.com/Coffeehedake/yarg-song-server` (GitLab mirror 10)
`yarg/`	`gitlab.badassium.com/fatalexception/yarg` (project 55)	`github.com/Coffeehedake/yarg` (GitLab mirror 11)
`yarg-remote/`	`gitlab.badassium.com/fatalexception/yarg-remote` (project 57)	`github.com/Coffeehedake/yarg-remote` (GitLab mirror 13)

All three mirrors are one-way, all branches, divergent refs not kept. 10 and 11 were verified end to end on 2026-09-05; 13 on 2026-09-10. `Coffeehedake/yarg` is a real GitHub **fork** of `YARC-Official/YARG` — the only shape GitHub accepts an upstream pull request from.

- `yarg-remote/` **is the phone app** (Capacitor + React + TypeScript) for the in-game remote song queue, which is upstream issue #860. Its server is inside the fork, at `yarg/Assets/Script/Integration/RemoteQueue/` — so a change to a route or a DTO there is a change to this repo's contract, and `yarg-remote/src/types.ts` is a transcription of `RemoteQueueBridge.cs` rather than a design of its own. Read `yarg-remote/CLAUDE.md` first.

A NEW MIRROR DOES NOT RUN UNTIL SOMETHING PUSHES. Mirror 13 was created after the first push to project 57 and therefore sat at `update_status=none` with an empty `last_error` — which looks exactly like a healthy mirror that has simply never had anything to do. Force it with `POST /projects/:id/remote_mirrors/:mirror_id/sync` (GitLab 16.7+). The project-level `POST /projects/:id/mirror/push` is for PULL mirroring and 404s here, which reads like a permissions problem and is not one.

And verify the far end with `git ls-remote`, not the GitHub REST API. Straight after mirror 13 reported `finished`, `GET /repos/Coffeehedake/yarg-remote/commits/main` answered **409 "Git Repository is empty"** while `git ls-remote` showed `refs/heads/main` at the correct commit. The REST repository record had not caught up with a push that had already landed. Believing the API there would have meant re-running a sync that had worked — the same family as "a directory listing under a service's storage is not that service's inventory", further down this file.

Rules:

- **Push to `origin` (Vault2 GitLab) only.** GitHub is a downstream, force-overwritten mirror — never push to it directly, never commit to it, never open the PR there first.
- **LFS does propagate.** An earlier note here claimed GitLab push mirroring drops LFS objects. That was wrong and has been measured: on GitLab 18.11.3, a fresh 1 MB LFS object pushed to a non-fork GitLab project arrived in GitHub's own LFS storage and downloaded with a matching SHA-256. Nothing special is needed when client work adds a `.png`, `.jpg`, `.exr`, `.fbx` or `.ttf` — the five patterns YARG's `.gitattributes` tracks.
- **The mirror force-overwrites, and the GitHub side is a fork.** Anything done directly on GitHub — a branch pushed there, a merge into the fork's `dev` — is clobbered on the next sync. To open an upstream PR: create the branch on GitLab, let it mirror, then open the PR from the mirrored branch, and do not touch it on GitHub afterwards.
- `yarg/` additionally carries an `upstream` remote pointing at `YARC-Official/YARG`. **Upstream PRs target `dev`, never `master`** — upstream will refuse `master` PRs outright.
- **`yarg/` IS now cloned** (2026-09-07), at `dev`, with `upstream` pointing at `YARC-Official/YARG`, submodules initialised and LFS pulled — 4,815 files, 0.25 GB. The `YARG.Core` submodule sits at upstream `6c0923e` (`028969a` until the 2026-10-30 rebase, `8ae7718` until the 2026-11-02 merge that brought the fork current — see the ROADMAP).

YARG.Core builds and tests without Unity. It is `netstandard2.1` with a plain `Microsoft.NET.Sdk` csproj, so `dotnet build YARG.Core.sln` and `dotnet test YARG.Core.UnitTests` both work — which matters because the Phase 3 seam (`SongEntry`) lives in YARG.Core, not in the Unity project. That required the **.NET 10 SDK**: the unit-test and benchmark projects target `net10.0`, and ENG-1 had only 8.0.424. 10.0.400 is now installed side by side in `C:\Program Files\dotnet`.

Baseline, measured 2026-11-02 at `6c0923e`: 750 tests, 742 pass, 6 fail, 2 skipped. The six are lip-sync phoneme tests (`AY_MapsToEatThenIf`, `ER_MapsToEarth`, `EY_MapsToOxThenIf`, `NG_MapsToNew`, `OW_MapsToOxHeld`, `OY_MapsToOxThenEat`) and they are **upstream's**, not ours — inferred, not re-run against a pristine upstream checkout: none of our commits touch YARG.Core and the submodule pointer equals `upstream/dev`'s. (At `8ae7718` it was 745

tests, 737 pass — five tests were added by the two commits the 11-02 merge brought in, and all five pass. At `028969a` it was 547 tests with the one failure `PossibleInstrumentsForSong_SixFretIncludesFiveFretInstruments`; that one no longer fails.)

A SEVENTH failure appears intermittently, and the count is only stable if you know about it. `PreviewContextTests.WaitForCompletionAsync_AllowsFadeOutAfterCancellation` failed on one of five runs on 2026-11-02 — the one that shared the machine with other work. **Measured properly on 2026-11-03: the test alone, 20 consecutive runs on an idle machine, 19 pass and 1 fail — a 5.0% failure rate with nothing else competing for the CPU.** It is a wall-clock race rather than a mystery: the test cancels a context with `fadeDuration: 0.05`, waits `Task.Delay(10)`, and asserts the wait task is *not* yet complete, so any scheduling hiccup over about 40 ms lets the 50 ms fade finish and flips it. Upstream's test, upstream's race. **Run the suite more than once before calling a seventh failure a regression**, and do not quietly record 7 as the baseline.

The two counted skips are `FullScan()` and `QuickScan()`, which want a real song library and are worth revisiting since this project has one. Two further tests (`Canonical20260801_*`) *print* as skipped without being counted, so four skip lines and a total of 2 both appear; 742 + 6 + 2 = 750 adds up. Not investigated.

Do not report "YARG.Core is green" as a baseline; it is not, and a new failure would hide in that assumption.

The Unity Editor IS now installed at `C:\Program Files\Unity\Hub\Editor\6000.3.5f2`, matching the pin in `ProjectSettings/ProjectVersion.txt`. *(This paragraph used to say it was not installed and not needed yet; that is stale.)* **Launch it with** `Win32_Process.Create` — see the gotcha below, or every run dies blaming anti-virus.

`ProjectSettings/ProjectSettings.asset` **must be** `git checkout --` **reverted after EVERY editor run**, including ones that die: Unity re-adds a VisionOS icon block whether or not the run got anywhere.

`ProjectSettings/QualitySettings.asset` **too, and that one is NOT in the note below.** A batchmode run on 2026-11-03 flipped `vSyncCount: 0` to `1` in it. One line, no content, and nothing announces it — so `git add -A` after a probe sweep commits a rendering setting change to a fork intended for upstreaming. Check `git status` after an editor run rather than reverting only the file this file used to name.

A PLAYER build leaves a third thing, and it is UNTRACKED rather than modified: `Assets/Settings/AddressableAssetsData/link.xml` and its `.meta`. Addressables generates them during the build and `.gitignore` does not cover them, so they sit in `git status` under "untracked" where a `git add -A` sweeps them up. They are build output; leave them alone. - **You do not need a GitHub credential for either repo, and should not go looking for one.** GitLab owns the mirror credential and pushes for us. Measured 2026-09-06: mirror 10 on project 53 last succeeded at `20:42:42`, the same minute as that push to origin, with an empty `last_error`; mirror 11 on project 55 is enabled and last succeeded 2026-09-05. A GitHub PAT is only needed for something GitLab cannot do on our behalf — opening a pull request against `YARC-Official` through the API, say. - **A dead mirror is silent, and this is where it shows.** Nothing alerts if the mirror's stored credential expires: pushes to origin keep succeeding, and GitHub simply stops moving. The only signal is the API, so check it rather than assuming the mirror ran — same failure shape as the Unraid disk alerts that have not fired since March 2025.

```
powershell $tok = (& pwsh -NoProfile -File 'C:\dev\_environment\Get-DevCredential.ps1' -Target 'dev:gitlab-ce-pat' | Out-String).Trim() foreach ($proj in @(53,55)) { (Invoke-RestMethod "https://gitlab.badassium.com/api/v4/projects/$proj/remote_mirrors" ` -Headers @{ 'PRIVATE-TOKEN' = $tok }) | ForEach-Object { "project ${proj}: enabled=$($_.enabled) status=$($_.update_status) last_success=$($_.last_successful_update_at) err='$($_.last_error)'" } }
```

Note `$proj`, not `$pid` — **`$PID` is a read-only automatic variable in PowerShell** and assigning to it fails the whole loop with an error that points at the `foreach`, not at the name. Same family as `r` (`Invoke-History`) and `h` (`Get-History`). - Vault2 GitLab-CE API token: `op://fallout-automation/26tovxp2kthsdekw7dql4zeqzy/credential`, or `dev:gitlab-ce-pat` in Windows Credential Manager — **prefer Credential Manager for anything polled**, since `op` reads are rate-limited per service account. Enable the headless 1Password service account first: `. C:\dev\_environment\enable-headless-op.ps1`. Never put a token in chat, in a commit, or in a file. - **GitHub PAT, when one is genuinely needed: `op://MCP/xzsly5bgd2orqj2xy524txnfiu/credential` — vault `MCP`, item titled exactly `GitHub PAT`.** Same item the credentials list already calls "Coffeehedake GitHub".

The trap is in `Home-Lab`, not in the name. That vault holds `GitHub PAT (pc-deploy hipot-autobuild read-only)` — **read-only, and belonging to another project** — whose title is the closest match to a bare "GitHub PAT" search. Grabbing it gets a token that authenticates fine and then fails on the first write, which reads as a permissions bug in whatever you were doing rather than as the wrong token. `Home-Lab` also holds `GitHub MCP PAT (Claude Desktop, ENG-1)`, which is a third, separate credential for the Claude Desktop MCP server.

So: **address it by ID in vault `MCP`**, and do not search across vaults by title. Note that `op item list` with no `--vault` rate-limits the whole service account for the better part of an hour, so a title search is expensive as well as ambiguous.

Stack decisions

- **Server: Go.** Single static binary, trivial cross-compile to arm64 (Pi), macOS and Windows, tiny container. See `yarg-song-server/docs/ADR-001-server-architecture.md`.
- Go means **YARG.Core is not available** — the server reimplements the parts of the song format it needs. **Before writing any parser code, read `yarg-song-server/docs/SOURCES.md`**: much of this is documented on the official wiki and by TheNathannator, and deriving `song.ini` from source when a better spec existed cost this project four real defects.
- **The server is a content source, not a game feature.** It emits ordinary `.sng` files that an unmodified YARG can already read. Client-side integration comes later and separately.

Hard constraints

- **Never implement CON / mogg decryption.** Upstream's `CONTRIBUTING.md` puts "CON Decryption" in the Out of Scope tier ("your PR will immediately be denied"), and it carries real DMCA 1201 exposure. The server refuses `.con` / `_rb3con` / `.pkg` on ingest with an explicit message.
- **Never generate `songcache.bin`.** Not because it is unreadable — it is a plain binary file and an external tool could write one. Because it stores **absolute local paths**, so a cache built on a server is meaningless to a client; because `CACHE_VERSION` is a date stamp checked with no compatibility window and no migration, over a field layout that is `internal` with no version of its own; and because getting it wrong fails *silently* — YARG just rebuilds and the tool appears to have worked. The server ships its own JSON catalog instead.
- **Never distribute copyrighted audio.** The eventual auto-charting feature must be able to emit chart/vocal tracks as a package separate from the music, so charts can be shared without audio.

Conventions

- Docs are written in the same change as the code, never batched for later.
- Branded PDFs use the **FatalException** brand (personal project) — never Juniper.
- Verify the local repo is actually up to date against origin before starting work AND again before committing; the other machine pushes mid-session.
- **THE TWO MACHINES ARE INDEPENDENT. Nothing syncs between them.** Syncthing was retired on 2026-09-06 and nothing replaced it: on ENG-1 the logon task is disabled, no process runs and nothing listens on 8384/22000; the Vault2 container sits in `Created` and has never started. So `git push` **to origin is the only thing that moves work between ENG-1 and r7** — an uncommitted file, or a file in no repo at all, exists on exactly one machine.

Two consequences that bite immediately:

- **This file is inside the `yarg-song-server` repo for that reason.** It used to live at the folder root, outside any repo, on the assumption that the folder itself synced. The moment that assumption died the file reached nowhere. Anything a future session must read has to be *in a repo that is pushed* — the folder is not a transport.
- The Cowork project *card* does not sync either, and never did. Recreate it per machine and point it at this folder. These instructions live here precisely so nothing is trapped in the card.

Branding

FatalException, not Juniper — this is a personal GitLab project. Render docs with the central builder, never a project-local copy:

```
python "C:\dev\fatalexception-brand-kit\scripts\build-pdfs.py" "C:\dev\YARG - Open Source Contributions\yarg-song-server"
```

Regenerate the affected PDF in the same change as the markdown, so the two never drift.

Tooling installed for this project

All per-user under `%LOCALAPPDATA%\Programs\`, no elevation, each deletable as one folder:

What	Where	Why
YARG v0.15.0	`Programs\YARG\YARG.exe`	The oracle. The only thing that can say whether a package we produced is really acceptable
SngCli v0.3.0	`Programs\sngcli\win-x64\SngCli.exe`	The reference `.sng` encoder/decoder
Go 1.27.0, mingw-w64 GCC 16.2.0	`Programs\go`, `Programs\mingw64`	Toolchain; the GCC is what makes `go test -race` work at all

What	Where	Why
Unity 6000.3.5f2	`C:\Program Files\Unity\Hub\Editor\6000.3.5f2`	The fork's pinned editor. Installed since the note above was written — but see the UPM gotcha, it does not currently start
Node 24.15.0, npm 11.12.1	system	`yarg-remote`. **Install with** `--include=dev` — see below

Running the oracle — worth knowing, because it has found bugs no unit test did:

```
go run ./cmd/mkcorpus -out $env:USERPROFILE\yarg-test\corpus
# point YARG at it: edit SongFolders in
#   %USERPROFILE%\AppData\LocalLow\YARC\YARG\release\settings.json
# delete songcache.bin beside it, launch YARG, wait ~45s, then read badsongs.txt
```

`badsongs.txt` is YARG's own verdict on every song it refused, and the song cache's readable strings show which titles it accepted. That loop found three real bugs our tests had passed.

Gotchas

Unity batchmode must be launched with `Win32_Process.Create`

Not `Start-Process`, and the failure blames the wrong thing:

```
[Package Manager] Could not connect to IPC stream "Upm-<pid>" after 30.0 seconds.
[Package Manager] Failed to start the Unity Package Manager local server process ...
                  blocked by Windows Defender or any other anti-virus configuration
```

It is not anti-virus. A process started from a Cowork bridge call inherits a **job object**, and that job object kills Unity's Package Manager *child*. Detach the launch and it works:

```
$cmd = '"' + $unity + '" -batchmode -nographics' +
       ' -projectPath "' + $proj + '"' +
       ' -logFile "' + $log + '"' +
       ' -executeMethod Editor.RemoteQueueHost.RunLocal'
Invoke-CimMethod -ClassName Win32_Process -MethodName Create -Arguments @{ CommandLine = $cmd }
```

Quote the project path inside the command line — the folder name has spaces, and unquoted it becomes three arguments that Unity resolves against `C:\Windows\System32`.

`-noUpm` is not a workaround. It gets past this and then dies on `TMP_Text`: the packages it disables are real dependencies of the fork.

This cost an hour on 2026-09-10, and it was already written down — in the Cowork project-state doc, in almost these words, which was not read first. Five failed launches, a "blocker" filed against the whole editor track, and a wrong cause (Defender) published in a commit message and a roadmap entry, all retracted the moment the existing note was

found. The rule this is an instance of is already further down this file as *"the answer is often already in the Arbiter inbox"* — it applies to the project's own documents just as much.

`%LOCALAPPDATA%\Unity\Editor\upm.log` is the evidence either way: a launch that fails this way leaves **no entry at all**, because the child never ran.

`npm install` here installs NO build tooling and exits 0

`npm config get omit` returns `dev` on this machine, from the **global** npmrc (`%APPDATA%\npm\etc\npmrc`), not from any project. So a plain `npm install` in `yarg-remote` reports "added 10 packages", exits 0, and leaves no vite, no typescript and no vitest — 8 directories in `node_modules` instead of 316. Nothing says the word "dev".

The failure surfaces much later, as `'vite' is not recognized`, which reads like a broken `package.json` or a corrupt install. **Use `npm install --include=dev`.**

The folder name has spaces

`YARG - Open Source Contributions` contains `-`. Any tool invoked with an unquoted path splits it into separate arguments — this has broken the PDF builder twice, resolving a lone `-` against the home directory and reporting a confusing `C:\Users\ENG2\-` not-found that reads like the ENG-1 wrong-machine-path quirk but is not. **Always quote the path**, including inside `Start-Process -ArgumentList`, which does no quoting of its own.

Never give a PowerShell helper a single-letter name

The alias wins. `r` is `Invoke-History` and `h` is `Get-History`, so a `function R` fails every call with "*Cannot locate the history for command line …*" and a `function H` with "*Cannot bind parameter 'Id'*". Both happened here, days apart, and both read as a bug in the thing being called. Assume every letter is taken; name helpers `Get-SngHashes`, not `H`.

`$_` and `$env:` are mangled inside an inline `pwsh -Command`

The outer shell expands them before `pwsh` ever sees the string, so a pipeline using `$_` becomes a parser error about a missing operand. **Write a `.ps1` and run `pwsh -NoProfile -File`.** This is not an occasional annoyance — every inline command with a `ForEach-Object` block fails this way.

PowerShell variable names are case-insensitive

`$C` and `$c` are the same variable. A `foreach ($c in $cases)` loop silently overwrote a `$C` holding the corpus path, and the resulting failure looked exactly like a bug in the Go code — `open .: The system cannot find the path specified` — for two tool calls. Use distinct, wordy variable names in any loop.

A long child process dies with the bridge

A process started from a Cowork session is killed when a bridge call times out, which silently cancelled a `winget` install mid-download. For anything running longer than about a minute, register a **scheduled task**; it is detached and survives.

The device-bridge Linux VM is a real Linux, and that is worth using

Windows cannot see POSIX-only concurrency defects: an open handle blocks `os.Remove`, so a goroutine racing to delete a file another goroutine is about to open simply fails and the race disappears. That masked a real 500 in `packcache` through five clean stress runs, and Linux CI found it on the first pipeline. **Go 1.25.1 is installed in the bridge VM at `$HOME/go`** for exactly this; run concurrency and filesystem-race work there before believing a green.

Two things about that VM that cost time:

- **A background job does not survive the call that started it.** Each `device_bash` call is a fresh `bwrap --unshare-pid --die-with-parent` namespace, so `nohup … &` dies the moment the call returns and the log file is left empty — which reads exactly like a job that is still running. Run long work in the **foreground** with the timeout raised; the ceiling is 180 s.
- **`gofmt -w` on the mounted folder can leave its temp file behind**, named `<file>.go.<digits>`. It shows up as untracked in `git status` and will be committed by a `git add -A` that nobody looked at first.
- **`device_bash` cannot delete on the mount** ("Operation not permitted"), so cleaning those temp files means PowerShell — and that is where the real trap is. **`Get-ChildItem -Filter "*.go.*"` MATCHES `api.go`.** `-Filter` is passed to the filesystem and evaluated with 8.3 short-name semantics, where a trailing `.*` also matches "no extension after the dot". Measured 2026-09-08: `Get-ChildItem -Filter "*.go.*" | Remove-Item` aimed at one leftover `check.go.4044019482262576786` **deleted every .go file in the directory** — api.go, check.go, web.go, all three test files. Tracked files came back from `git restore`; the new untracked file did not exist anywhere but that directory, and only survived because a copy happened to be in the session's scratch space.

Filter with `Where-Object` on a real regex instead, and delete by literal path:

```
powershell Get-ChildItem -LiteralPath $dir -File | Where-Object { $_.Name -match '\.go\.\d+$' } | ForEach-Object { Remove-Item -LiteralPath $_.FullName -Force }
```

The general rule this is an instance of: **`-Filter` is not a glob and not a regex.** When a delete is involved, list first, read the list, then delete by name. - **Never ask "is this directory empty?" with `ls -la … | head`.** `total N`, `.` and `..` are three lines before any content, so `head -3` shows exactly what an empty directory shows. Measured 2026-09-08, expensively: a registry storage directory read that way was reported as empty, and the conclusion drawn from it — that every image in the GitLab-CE registry might be gone — went into a handoff, two project messages and a status report before being retracted. It held **84 GB** and twelve repositories, starting on line 4. Use `ls -A | wc -l`, or `du -sh` when the answer should be a size. - **Git run from the mount cannot remove its own `.git/index.lock`** (same "Operation not permitted"), so a stale zero-byte lock is left behind and blocks the next commit. Check for a running `git` process, then remove it from PowerShell.

Install Unity Hub from Unity's own installer, never the MSIX

`winget install Unity.UnityHub` serves the **MSIX** package, and the MSIX Hub cannot license or launch an Editor. MSIX virtualises the app's writes into `%LOCALAPPDATA%\Packages\UnityTechnologies.UnityHub_…\LocalCache\`, but the Editor is a separate **unpackaged** process that reads the real `%APPDATA%` and `%LOCALAPPDATA%`. So the Hub signs in and writes `UnityEntitlementLicense.xml` into the container, the Editor looks in `%LOCALAPPDATA%\Unity\licenses`, finds nothing, and exits with code 1. The visible symptom is a different one — "*Failed to resolve project template: … `%APPDATA%\UnityHub\Templates\…tgz` was not found*" — because the same split hides the 338 MB template the Hub just downloaded.

Use `https://public-cdn.cloud.unity3d.com/hub/prod/<version>/UnityHubSetup-<version>-x64.exe`. The bare `.../hub/prod/UnityHubSetup.exe` URL that is all over the internet is **404** now; `winget show --id Unity.UnityHub` is a reliable way to learn the current version number even though its installer is the wrong package.

Three related traps, all met the same afternoon:

- `winget install` **printed** `Successfully installed` **and installed nothing.** The classic installer is machine-scope and the bridge shell is not elevated, so it exits 0.
- `Start-Process -Verb RunAs` **did not elevate from a bridge call.** The retry got a truthful `0x80070005 Access is denied`. Do not plan on elevating from here; hand Jay the installer.
- `Test-Path "$env:ProgramFiles\Unity Hub"` **is the wrong check for an MSIX.** It never exists. `Get-AppxPackage` is the check — searching the filesystem and concluding "the install failed" led to uninstalling a package that was fine.

And the Hub CLI needs `$env:ALLUSERSPROFILE = 'C:\ProgramData'` set explicitly, or it dies with `Unable to resolve config folder: ALLUSERSPROFILE is not set` — the bridge shell's environment is stripped.

Install the Editor version the project pins, from `ProjectSettings/ProjectVersion.txt` — `6000.3.5f2` / changeset `3fa8bc678cb0` for the `yarg` fork. The Hub's default offer is the newest LTS, and opening the fork with it silently migrates the project, which is the last thing a repo intended for upstreaming should do. Point the editor install path at `%LOCALAPPDATA%\Programs\Unity\Hub\Editor` (the house convention, and it avoids the admin that the default `Program Files` path would need) — **note this applies to CLI installs only**: an install started from the Hub's GUI with admin lands in `C:\Program Files\Unity\Hub\Editor` regardless, which is where 6000.3.5f2 actually is. Check both roots before concluding an editor is missing.

```
$env:ALLUSERSPROFILE = 'C:\ProgramData'
& "$env:ProgramFiles\Unity Hub\Unity Hub.exe" -- --headless install-path -s
"$env:LOCALAPPDATA\Programs\Unity\Hub\Editor"
& "$env:ProgramFiles\Unity Hub\Unity Hub.exe" -- --headless install --version 6000.3.5f2 --changeset 3fa8bc678cb0
```

Building the `yarg` fork

Verified end to end on 2026-09-07: **the fork imports and compiles clean under Unity 6000.3.5f2, batchmode, exit code 0, 121 assemblies, zero** `error CS` (20 warnings). About seven minutes for a cold import; `Library/` ends at 2.5 GB.

Restore the NuGet packages FIRST, or you get 270 errors that look like broken code

`Assets/packages.config` lists **16 NuGet packages** restored by NuGetForUnity into `Assets/Packages/`, which is gitignored and therefore **absent in a fresh clone**. Without them a batchmode import fails with 270 `error CS` — `Melanchall` (DryWetMidi), `Cysharp` (ZString), `Utf16ValueStringBuilder`, `Microsoft.VisualStudio`, and so on. Every one is a missing dependency and none of them is about the code, but the volume reads like a broken checkout.

```
dotnet tool install --global NuGetForUnity.Cli --version 4.5.0
$env:DOTNET_ROLL_FORWARD = 'LatestMajor'      # see below
& "$env:USERPROFILE\.dotnet\tools\nugetforunity.exe" restore "C:\dev\YARG - Open Source Contributions\yarg"
```

`DOTNET_ROLL_FORWARD` **is not optional on ENG-1.** The tool targets .NET 9; ENG-1 has 8.0.30 and 10.0.11 and nothing in between, so it refuses to launch with a framework-not-found error that reads like a broken tool install. Rolling forward runs it on 10 and needs no extra runtime.

The import itself

```
& "C:\Program Files\Unity\Hub\Editor\6000.3.5f2\Editor\Unity.exe" `
    -batchmode -quit -nographics `
    -projectPath "C:\dev\YARG - Open Source Contributions\yarg" `
    -logFile "$env:TEMP\yarg-unity-import.log"
```

Do NOT pass `-accept-apiupdate`**.** Without it, Unity's API updater warns instead of running, so it can never silently rewrite source in a fork we intend to upstream. With the editor version matching the pin it has nothing to do anyway.

Do NOT pass `-noUpm` **to get around a Package Manager failure.** It resolves no packages, so the compile that follows is missing TextMeshPro and every other package dependency and reports errors that have nothing to do with the code under test. A run with it is not comparable to a run without it.

Unity writes to `-logFile` rather than stdout, so it has none of the `EPIPE` fragility the Unity Hub CLI has. Launching it, though, is the part that bites:

Launch Unity with `Win32_Process.Create`, not `Start-Process`

Measured 2026-09-07. Unity started from a bridge PowerShell call — `& $unity` or `Start-Process` alike — dies after 30 s with:

```
[Package Manager] Could not connect to IPC stream "Upm-<pid>" after 30.0 seconds.
[Package Manager] Failed to start the Unity Package Manager local server process.
```

The message blames anti-virus and that is a red herring. Measured instead: `UnityPackageManager.exe` runs fine on its own (`--version` → `v9.21.3`), Defender has no ASR rules and Controlled Folder Access is off, there is 1.1 TB free, and `%LOCALAPPDATA%\Unity\Editor\upm.log` gets **no new entry at all** — polling `Win32_Process` for the whole 30 s window shows the child is never created. The editor is not failing to talk to UPM; it is failing to *spawn* it, because a process tree started from a bridge call inherits a job object that will not let it.

The fix is to have something outside that job create the process:

```
$cmd = '"C:\Program Files\Unity\Hub\Editor\6000.3.5f2\Editor\Unity.exe" ' +
       '-batchmode -nographics -projectPath "C:\dev\YARG - Open Source Contributions\yarg" ' +
       '-logFile "' + $log + '" -executeMethod YARG.Editor.SettingsRowProbe.Run'
$r = Invoke-CimMethod -ClassName Win32_Process -MethodName Create -Arguments @{CommandLine = $cmd}
```

WMI creates it from `WmiPrvSE`, so it is outside the bridge's tree entirely. The UPM child appears within 25 s and the run completes normally.

And hide the window, or it steals focus every time — added 2026-11-03. `Create` shows a console, and with the `cmd.exe` wrapper that carries the environment variables (it has to: the WMI service does not pass this shell's environment to the child) every launch pops a window and **takes keyboard focus**. One run is a flicker. A sweep of

seventeen probes, while Jay was working at the machine, was seventeen interruptions mid-sentence — he asked for it to stop, and he was right to.

`ProcessStartupInformation` takes a `Win32_ProcessStartup` whose `ShowWindow` is an `SW_*` constant; `0` is `SW_HIDE`. The job-object breakaway is unaffected, because the process is still created by the WMI service rather than by this shell:

```
$startup = New-CimInstance -ClassName Win32_ProcessStartup -ClientOnly -Property @{
    ShowWindow = [uint16] 0          # SW_HIDE
}
Invoke-CimMethod -ClassName Win32_Process -MethodName Create -Arguments @{
    CommandLine               = 'cmd.exe /c "<wrapper>.cmd"'
    ProcessStartupInformation = [CimInstance] $startup
}
```

Use it for anything launched more than once, and for anything launched at all while somebody is using the machine. "Headless" means no window as well as no graphics; `-batchmode -nographics` only covers Unity's own. This also replaces the older "register a scheduled task" advice for Unity specifically — same breakaway, far less ceremony — though a scheduled task is still right for anything that must outlive the whole session.

Quote the project path inside `-ArgumentList`. `Start-Process -ArgumentList @(..., $p, ...)` splits `C:\dev\YARG - Open Source Contributions\yarg` on its spaces and Unity exits with `Couldn't set project path to: C:/Users/ENG2/C:/dev/YARG`. The same applies to the WMI command line above, which is why every path in it is quoted.

Opening the project dirties one tracked file, harmlessly

Unity 6000.3 adds a `VisionOS` icon block to `ProjectSettings/ProjectSettings.asset` — seven lines, no content. It is not ours and it is not upstream's; **revert it rather than committing it**, and expect it back every time the editor opens:

```
git checkout -- ProjectSettings/ProjectSettings.asset
```

Everything else Unity writes (`Library/`, `Temp/`, `Logs/`, `Assets/Packages/`) is gitignored.

Run git on Windows, not in the bridge VM

`core.autocrlf` is `true` on ENG-1, and `git diff` from the VM reports every CRLF file as fully rewritten — 25 files and 3,840 insertions, none of them real. Read and edit in the VM; run **git** on Windows.

A directory listing under a service's storage is not that service's inventory

2026-09-08. Checking whether CI had published an image for `main`'s head, I listed the registry's `_manifests/tags` directory on disk inside the GitLab-CE container. It returned 69 tags and did not include the one I was looking for, whose pipeline had succeeded 90 minutes earlier. The GitLab API returned 70 and did include it.

This is the same shape as `ls -la <dir> | head -3` "proving" a directory was empty, and as `-Filter "*.go.*"` matching `api.go`: a plausible instrument, pointed at the right place, answering a slightly different question than the one asked. A registry's on-disk layout is an implementation detail with caching and write ordering behind it; the API is

the thing with a contract.

The rule that keeps coming out of these: **when a service exposes an API for a fact, do not read that fact off its filesystem** — and when you do read the filesystem, say so in the finding, so the claim carries its instrument with it.

The fix for our red pipeline had been published two hours before we read for it

2026-09-08. This project's `container-image` job failed three times with `blob unknown to registry`. Between the second failure and the third, the session that owns GitLab-CE measured the cause and broadcast the mitigation — `--provenance=false` on the multi-arch `buildx` push, 6 pass / 0 fail against 3 pass / 3 fail — to `ENG-1`, which is us. We failed again **two hours later** without having read it, and instead spent that time re-deriving the incidence from job traces.

The re-derivation was not wasted; a second project's failure rate over 24 hours is real corroboration, and it is what got sent back. But the sequence was backwards, and it cost a red pipeline that did not need to be red.

Read the Arbiter inbox before investigating anything that touches shared infrastructure — the registry, GitLab, vault2, the runner. Not only at session start: the useful message here arrived *mid-session*, in the middle of work that was going to hit exactly the failure it described. Another session having already solved it is the normal case on this machine, not a lucky one.

Second, smaller lesson from the same exchange: **a changing hash across retries proves nothing on its own.** Three failures carrying three different blob shas were offered as evidence that the registry was losing a different blob each time. The conclusion held, but provenance attestations embed timestamps, so every rebuild of one commit yields different digests whether it fails or not. The argument was worthless even though the answer was right — which is the harder kind of mistake to notice.